Pontificia Universidad JAVERIANA Colombia

Prompts dinámicos con Jinja

Semana 2 · Plantillas, esquemas y trazas
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3

RAG y evaluación
Semanas 4–6

Model Context Protocol
Semana 7

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 2 · Prompts dinámicos con Jinja

- Por qué el prompt cambia con los datos
- Variables, condiciones y repeticiones
- Plantillas en archivo y `StrictUndefined`
- Instrucciones del equipo y datos del usuario
- Validar la salida con Pydantic
- Trazas y comparación A/B en LangSmith

Las ideas que traemos de la semana 1

Repaso

Antes de avanzar

1. El contexto influye en la respuesta del modelo.
2. Las instrucciones de sistema orientan su comportamiento.
3. Una plantilla permite reutilizar una instrucción.
4. Una salida con estructura facilita su uso, pero necesita revisión.

Lo que el ticket no dice

Conversación

El reto que dejamos abierto

El cliente escribe: «**ayer** me cobraron dos veces».

¿Qué debería responder el asistente? ¿Qué respuesta nos haría desconfiar?

Un ticket para pensar juntos

El caso de la semana

Caso sintético

1. Ticket T-2048: el cliente C-1042 reporta dos cargos iguales de 185.000 COP.
2. Los cargos son de su tarjeta débito, en la misma tienda en línea.
3. A veces el ticket trae la fecha; a veces dice «ayer»; a veces no dice nada.
4. Un ticket puede traer uno, dos o más cargos, incluso en otra moneda.

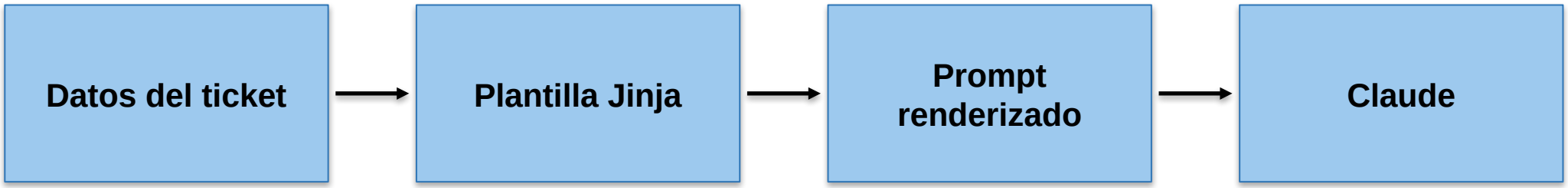
Pontificia Universidad JAVERIANA Colombia

Jinja

Un formato que se adapta a los datos

Una plantilla dinámica

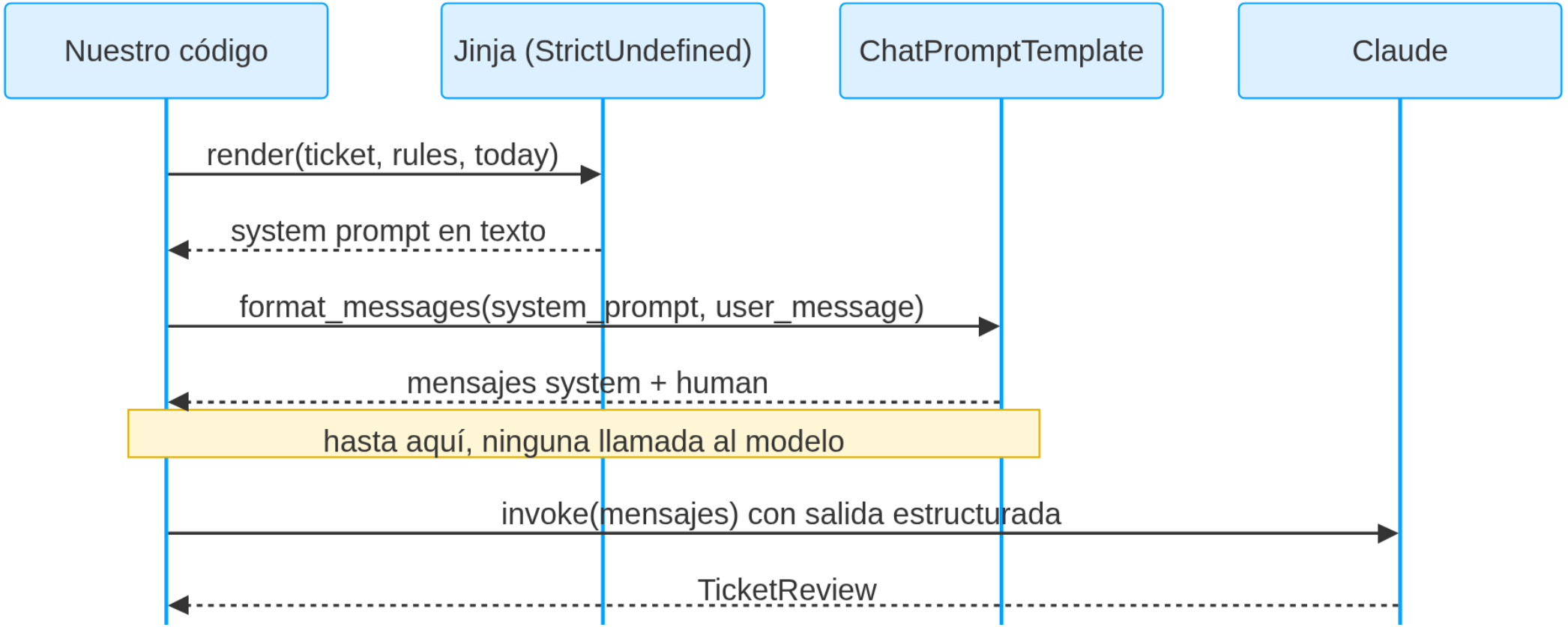
La intuición de Jinja



Renderizar la plantilla no llama al modelo ni consume tokens.

Del ticket al mensaje

Diagrama de secuencia



Renderizar no es llamar. Jinja produce el system prompt; el mensaje del cliente entra aparte, como dato; solo `invoke` llama al modelo.

Tres construcciones

La sintaxis que usaremos

Construcción	Sintaxis	En el ticket
Variable	`{{ ticket.customer }}`	El código del cliente
Condición	`{% if ticket.charge_date %}`	Con fecha o «no se indica»
Repetición	`{% for charge in ticket.charges %}`	Un renglón por cargo
Contador	`{{ loop.index }}`	Numerar las reglas de revisión

Más Jinja que vale la pena conocer

Filtros, bucles y espacios

Construcción	Ejemplo	Para qué
Filtro `default`	`{{ ticket.channel | default('app') }}`	Un valor por defecto explícito
Filtro `join`	`{{ products | join(', ') }}`	Una lista en una línea
Filtro `length`	`{{ ticket.charges | length }} cargos`	Contar sin escribir código aparte
`loop.first`, `loop.last`	`{% if not loop.last %}, {% endif %}`	Separadores dentro de un `for`
Comentario	`{# no se renderiza #}`	Notas para quien mantiene la plantilla
`trim_blocks`, `lstrip_blocks`	Opciones de `Environment`	Sin líneas en blanco por las etiquetas

La plantilla en un archivo

datasets/acme_ticket.jinja

Una sola fuente

El equipo revisa
y versiona el archivo
como cualquier código.

```
## Ticket {{ ticket.code }}
- Cliente: {{ ticket.customer }}
- Producto: {{ ticket.product }}
{% for charge in ticket.charges %}
- Cargo: {{ charge.amount }} {{ charge.currency }}
  en {{ charge.merchant }}
{% endfor %}
{% if ticket.charge_date %}
- Fecha de los cargos: {{ ticket.charge_date }}
{% else %}
- Fecha de los cargos: no se indica. No la inventes.
{% endif %}
```

Detectar datos que faltan

L03 · sección 6

StrictUndefined

«Fecha desconocida»
es un estado previsto.
Olvidar al cliente
es un error.

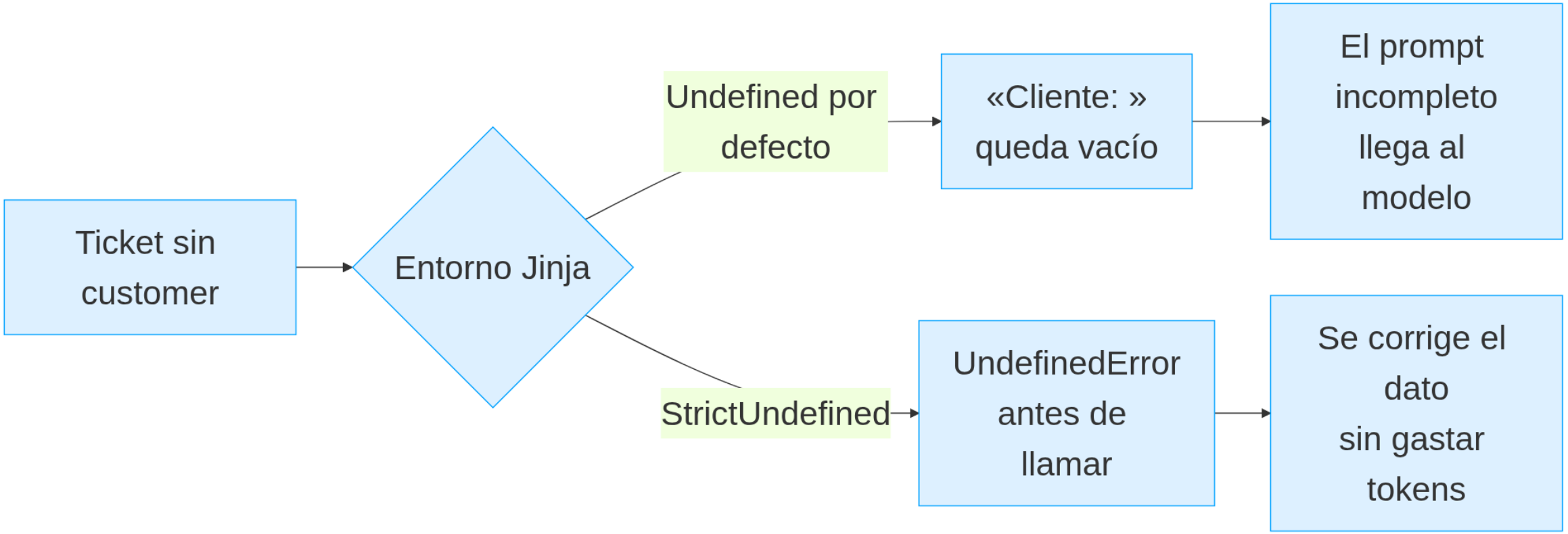
```
from jinja2 import Environment, FileSystemLoader, StrictUndefined

env = Environment(
    loader=FileSystemLoader("../datasets"),
    undefined=StrictUndefined,
    trim_blocks=True, lstrip_blocks=True,
)
template = env.get_template("acme_ticket.jinja")

# Sin StrictUndefined: «Cliente: » queda vacío
# Con StrictUndefined: UndefinedError antes de llamar
template.render(today=today, ticket=ticket_without_customer)
```

Undefined frente a StrictUndefined

Qué pasa cuando falta un dato



Fallar a tiempo. Con el modo por defecto el prompt incompleto llega al modelo y se paga; con `StrictUndefined` el error aparece antes.

Las instrucciones y los datos del usuario

L03 · sección 7

Cada texto tiene un papel

1. El equipo mantiene la plantilla y sus reglas.
2. El ticket aporta los datos del caso.
3. El cliente plantea la pregunta.
4. El mensaje del cliente se pasa como **dato**: `{{ ... }}` en su texto no se renderiza.

Dos riesgos distintos

Inyección de plantilla e inyección de instrucciones

Riesgo	Cómo ocurre	Qué lo reduce
Inyección de plantilla	El texto del cliente se renderiza como Jinja y ejecuta `{{ ... }}`	Pasar el texto como dato, nunca como plantilla
Inyección de instrucciones	El texto pide «ignora tus reglas» y el modelo lo lee	Roles separados, herramientas de solo lectura, validar la salida en código
Fuga de datos	El prompt incluye datos que la persona no debería ver	Enviar solo los campos necesarios

Una respuesta con campos definidos

L03 · sección 8

Salida estructurada

Pydantic valida
la forma, no la verdad.

```
class TicketReview(BaseModel):
    """Revisión de un ticket antes de abrir una disputa."""
    customer: str = Field(
        description="Código del cliente que reporta.")
    disputed_charges: int = Field(
        description="Número de cargos que pide revisar.")
    charge_date: str | None = Field(
        description="AAAA-MM-DD; None si no hay fecha exacta.")
    issues: list[str] = Field(
        description="Datos faltantes o ambiguos.")

review = model.with_structured_output(TicketReview).invoke(messages)
```

La estructura y la verdad

Validar la respuesta

Dos comprobaciones

El campo `disputed_charges` contiene 3. El cliente solo disputa los dos cargos de 185.000.

El esquema lo acepta. **¿Qué comprobación nos falta?**

Pontificia Universidad JAVERIANA Colombia

LangSmith

Observar cada llamada

La traza de una llamada

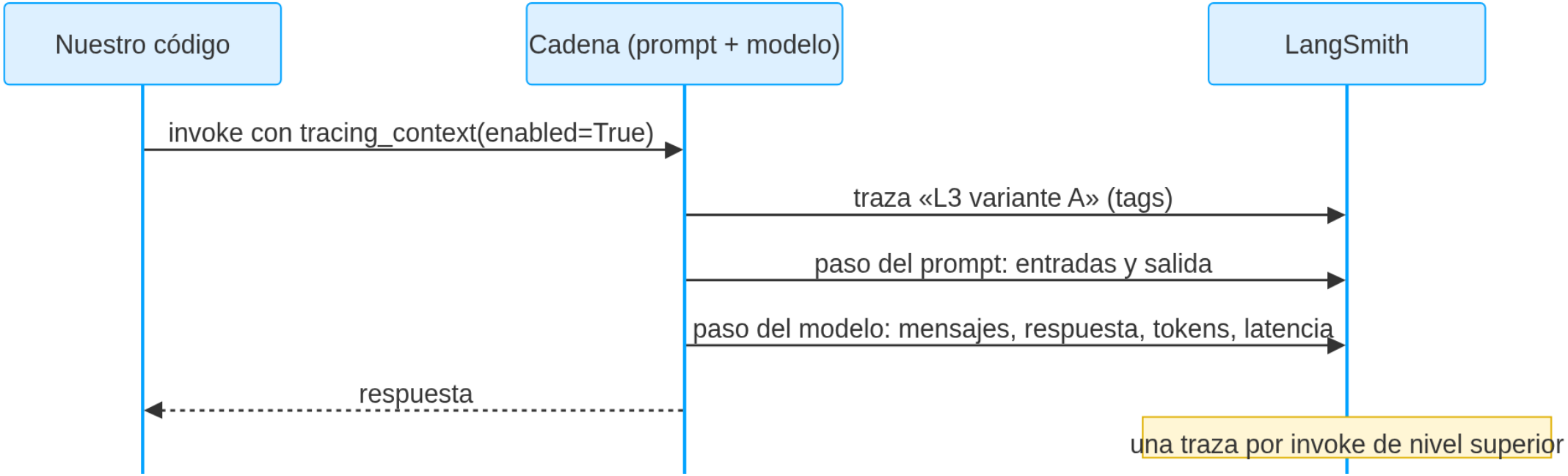
Observabilidad

La evidencia del recorrido

1. El prompt exacto que recibió el modelo.
2. La respuesta y su estructura.
3. El tiempo de la llamada y los tokens consumidos.
4. Etiquetas y nombres para encontrar cada variante.

Qué guarda una traza

Diagrama de secuencia



Una traza por invoke. Con `tracing_context(enabled=True)` cada `invoke` de nivel superior crea una traza con sus pasos; con el tracing apagado, nada sale del equipo.

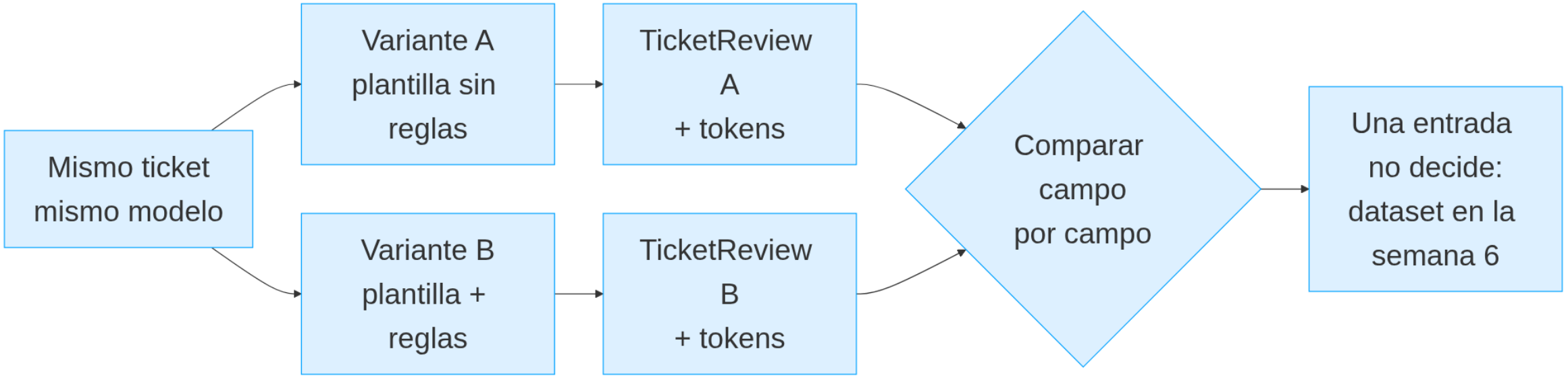
Dos variantes sobre el mismo ticket

L03 · sección 9

Variante	Qué cambia	Qué observar
A	Plantilla sin reglas adicionales	Datos faltantes y fecha ambigua
B	La misma plantilla con reglas de revisión	Si cambia la revisión y cuánto cuesta
Ambas	Mismo ticket, mismo modelo	Trazas `L3 variante A` y `L3 variante B`

Una comparación A/B

Lo que se puede y no se puede concluir



Mismo ticket, dos plantillas. Con un solo ticket se ven diferencias, no se demuestra cuál es mejor: eso pide un dataset y experimentos.

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L03-DynamicPrompting

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Fijar valores con `partial_variables` y pasar a Jinja.
2. Adaptar el prompt cuando falta la fecha o es ambigua.
3. Incluir cargos y reglas de longitud variable.
4. Detectar datos faltantes con `StrictUndefined`.
5. Validar la revisión con Pydantic y comparar dos variantes.

¿Cómo sabremos que funcionó?

Criterios de logro

La prueba del ejercicio

1. Con y sin fecha, el prompt cambia como esperamos.
2. Cada cargo aparece en el texto preparado.
3. Un dato obligatorio ausente produce un error antes de llamar.
4. La salida respeta el esquema y coincide con el ticket.
5. Podemos explicar por qué A y B cambian o coinciden.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Los datos pueden cambiar el prompt que construimos.
2. Conviene detectar faltantes antes de llamar al modelo.
3. La estructura facilita revisar; la fidelidad exige otra comprobación.
4. Las trazas ayudan a explicar lo que ocurrió.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 1	Plantillas y salida estructurada
AI Engineering, cap. 5	Prompt engineering y defensas
Documentación de Jinja	Plantillas, `if`, `for` y `StrictUndefined`
Documentación de LangSmith	Conceptos de observabilidad



Próxima semana

Tools y el primer **agente**: consultar la cartera de créditos de ACME.

Notebooks:

`L04-Tools.ipynb`

`L05-ToolsAPI.ipynb`

Sergio Alberto Mora Pardo - H1 2027

GitHub · LinkedIn · Repositorio del curso